

# 電腦圖學 Homework #1

## A Drawing Panel

黃俊源 01257128 2026-10-05

### Contents

|                                 |          |                             |           |                                            |           |
|---------------------------------|----------|-----------------------------|-----------|--------------------------------------------|-----------|
| <b>1 專案介紹</b>                   | <b>2</b> | <b>4.3 Stroke Rendering</b> | <b>9</b>  | <b>9.3 Layout、座標與Framebuffer的方向差異</b>      | <b>22</b> |
| 1.1 作業目標                        | 2        | <b>4.4 Fill Rendering</b>   | <b>11</b> | <b>9.4 讓操作可以預覽，也可以Undo/Redo</b>            | <b>22</b> |
| 1.2 系統概述                        | 2        | <b>4.5 座標系統</b>             | <b>11</b> | <b>9.5 粗線轉角的幾何例外</b>                       | <b>22</b> |
| <b>2 使用者操作說明</b>                | <b>2</b> | <b>4.6 Grid</b>             | <b>12</b> | <b>9.6 文字輸入與GFNT資料檢查</b>                   | <b>23</b> |
| 2.1 主畫面                         | 2        | <b>5 文字功能</b>               | <b>12</b> | <b>10 已知限制與未來改進</b>                        | <b>23</b> |
| 2.2 繪圖工具                        | 3        | 5.1 FreeGLUT 字型             | 12        | <b>11 心得</b>                               | <b>23</b> |
| 2.3 繪圖屬性                        | 3        | 5.2 Text Object 與文字輸入       | 13        | <b>A 自製 GUI Framework</b>                  | <b>24</b> |
| 2.4 Hierarchical Menu 操作        | 4        | 5.3 UTF-8 與中文字元支援           | 14        | A.1 設計動機與範圍                                | 24        |
| 2.5 快捷鍵與滑鼠操作                    | 4        | 5.4 自訂中文字型支援                | 14        | A.2 Application 與 Window                   | 25        |
| 2.6 視窗與對話框                      | 5        | <b>6 Framebuffer 與輸出</b>    | <b>15</b> | A.3 Element Tree 與 Layout System           | 25        |
| <b>3 系統架構與設計</b>                | <b>5</b> | 6.1 Color Buffer            | 15        | A.4 EventTarget、Bubbling 與 stopPropagation | 26        |
| 3.1 整體架構                        | 5        | 6.2 Display Restore         | 15        | A.5 Mouse Capture 與 Input State            | 26        |
| 3.2 Application 與 Window        | 5        | 6.3 圖片輸出                    | 16        | A.6 Multi-window Lifecycle                 | 26        |
| 3.3 Document、Scene 與 Shape      | 6        | <b>7 特殊貢獻摘要</b>             | <b>17</b> | <b>B GFNT 自訂點陣字型格式</b>                     | <b>26</b> |
| 3.4 Event 與 Input               | 6        | <b>8 成果展示與作業需求驗證</b>        | <b>18</b> | B.1 設計動機                                   | 26        |
| 3.5 Command History 與 Undo/Redo | 7        | 8.1 繪圖成果                    | 18        | B.2 Binary Layout                          | 27        |
| 3.6 Rendering Pipeline          | 8        | 8.2 進階功能成果                  | 19        | B.3 Loader 與驗證                             | 28        |
| <b>4 繪圖與渲染實作</b>                | <b>8</b> | 8.3 Requirement Checklist   | 19        | B.4 UTF-8 Decoder 與 Glyph Lookup           | 28        |
| 4.1 基本圖形                        | 8        | <b>9 開發問題與解決方法</b>          | <b>21</b> | B.5 Bitmap Rendering                       | 29        |
| 4.2 Interactive Drawing 與 Draft | 8        | 9.1 GLUT Callback 與多視窗狀態    | 21        |                                            |           |
|                                 |          | 9.2 從原始輸入整理出可用的 Event       | 21        |                                            |           |

# 1 專案介紹

## 1.1 作業目標

本次作業的目標為利用 OpenGL 與 FreeGLUT 實作一套互動式 2D Drawing Panel，讓使用者可以透過滑鼠與鍵盤在視窗中繪製基本幾何圖形與文字，並利用選單調整顏色、線寬、點大小及填滿模式等繪圖屬性。作業也要求程式處理 keyboard、mouse、window 與 motion event，並利用 Color Buffer 儲存及恢復畫面內容。

在完成基本要求的過程中，我希望程式不只是由大量 GLUT callback 與全域變數組成，因此將視窗、Event、繪圖物件、Document 與 Renderer 等功能分開。最後完成的程式命名為 **Graphoria**，使用 C++17、OpenGL 與 FreeGLUT 實作。

目前程式支援 Point、Pencil、Line、Rectangle、Ellipse、Polygon 與 Text 等工具，也能設定線條顏色、粗細、Join、Cap 與 Fill mode。另外加入 Undo/Redo、Grid、多視窗、文字輸入對話框、Color Buffer cache 與 PPM 圖片輸出等功能。尚未完成的部分集中在第 10 節說明。

## 1.2 系統概述

程式啟動時會先載入兩個 GFNT 字型，再建立主要繪圖視窗。畫面中央是可伸縮的 Canvas，底部狀態列顯示目前工具、滑鼠位置與畫布大小；按下右鍵則可開啟工具與屬性選單。滑鼠或鍵盤輸入進入程式後，會先轉成 Event，再由 Element tree 找出接收事件的元件。

繪圖時，工具會先建立暫時的 draft，讓使用者在拖曳過程中看到預覽。操作完成後，物件才會透過 Command 加入 Scene，所以 Undo/Redo 只需要處理 Document 的變化，不必知道每一種工具如何收集滑鼠座標。顯示時則依序畫出 Grid、Scene 內容與尚未完成的工具預覽。

# 2 使用者操作說明

## 2.1 主畫面

主畫面中央是 drawing canvas，底部狀態列顯示目前工具、Stroke/Fill 或文字字型、滑鼠在 Canvas 中的位置與 Canvas 尺寸。Canvas 預設使用 Pencil，Grid 預設為 Lines。圖 1 為主畫面的成果展示位置，內容包含 Canvas、狀態列與右鍵選單。

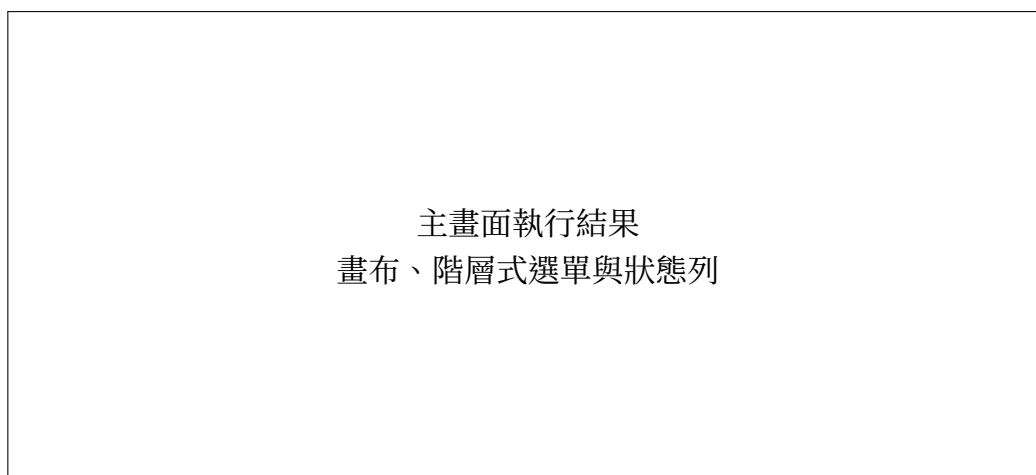


Figure 1: 繪圖程式主畫面

## 2.2 繪圖工具

Table 1: 目前工具與互動方式

| 工具               | 操作方式                                                             |
|------------------|------------------------------------------------------------------|
| <b>Point</b>     | 左鍵單擊，在指定位置建立點；大小由 Point size 決定。                                 |
| <b>Pencil</b>    | 按住左鍵拖曳，工具依距離過濾過密取樣點，放開時提交 Path。                                  |
| <b>Line</b>      | 左鍵拖曳決定兩端點；按住 Shift 限制為水平、垂直或 45 度。                               |
| <b>Rectangle</b> | 左鍵拖曳決定對角；按住 Shift 以較大的軸向差建立正方形。                                  |
| <b>Ellipse</b>   | 左鍵拖曳決定外接矩形；按住 Shift 建立正圓。                                        |
| <b>Polygon</b>   | 單擊增加頂點，游標移動時更新最後一個 draft 點；雙擊或 Enter 完成。Backspace 移除最後一點，Esc 取消。 |
| <b>Text</b>      | 點擊放置 anchor，接著在 modal input dialog 輸入文字。                         |
| <b>Select</b>    | 選單項目已建立，但 callback 為空，尚未完成。                                      |

## 2.3 繪圖屬性

Stroke 可設定八種預設顏色、寬度，以及 MITER、BEVEL、ROUND 三種 Join。端點則有 BUTT、SQUARE、ROUND 三種 Cap。Fill 有 OUTLINE、FILLED 與 ADVANCED 三種模式；其中 Advanced 會分開處理 Fill color 與自製 Stroke geometry。Point size 與 Stroke width 可由選單或中括號快捷鍵調整。Stroke、Fill 與 Text 選單中的 Custom... callback 目前為空，因此不列為已完成功能。

## 2.4 Hierarchical Menu 操作

在 Canvas 上按右鍵可以開啟主選單。File 可建立、開啟、儲存與輸出檔案；Edit 提供 Undo、Redo 與清空畫布；Tools 用來切換繪圖工具。Stroke、Fill、Point、Text 與 Grid 則分別調整筆畫、填色、點大小、字型及格線。常用功能也能使用下一節列出的快捷鍵，不必每次都打開選單。

## 2.5 快捷鍵與滑鼠操作

程式在內部以 Primary 表示作業系統慣用的主要修飾鍵。macOS 對應 **Command**，Windows 與 Linux 則對應 **Ctrl**。因此下表中的 **Primary+S**，在 macOS 要按 Command+S，在 Windows 或 Linux 要按 Ctrl+S。這個名稱只用來共用快捷鍵設定，畫面上實際按下的仍是各平台的按鍵。

Table 2: 程式中已綁定的全部快捷鍵

| 快捷鍵             | 功能                                          |
|-----------------|---------------------------------------------|
| Primary+Z       | Undo；若工具還在繪製 draft，會先取消這次未完成的操作。            |
| Primary+Shift+Z | Redo。                                       |
| C               | 清空目前 Scene。                                 |
| G               | 依序切換 Lines、Dots、None 三種 Grid 模式。            |
| Primary+N       | 建立新檔案；若內容尚未儲存，先顯示確認視窗。                      |
| Primary+Shift+N | 建立另一個 PaintWindow。                          |
| Primary+O       | 開啟檔案。目前檔案內容的反序列化尚未完成。                       |
| Primary+S       | 儲存目前檔案。                                     |
| Primary+Shift+S | 另存新檔。                                       |
| Primary+E       | 透過獨立 render window 輸出 PPM P6。               |
| Primary+C       | 關閉目前視窗；有未儲存內容時先要求確認。                        |
| Primary+R       | 要求視窗重新顯示，並重設鍵盤與滑鼠的輸入狀態。                     |
| 1               | 選擇 Pencil。                                  |
| 2               | 選擇 Line。                                    |
| 3               | 選擇 Rectangle。                               |
| 4               | 選擇 Ellipse。                                 |
| 5               | 選擇 Polygon。                                 |
| 6               | 選擇 Text。                                    |
| [               | Stroke width 與 Point size 減少 1 px，最小為 1 px。 |
| ]               | Stroke width 與 Point size 增加 1 px。          |
| Shift+[         | Stroke width 與 Point size 減少 5 px，最小為 1 px。 |
| Shift+]         | Stroke width 與 Point size 增加 5 px。          |

滑鼠左鍵負責繪圖，右鍵開啟 menu。Line、Rectangle 與 Ellipse 在拖曳時可按

住 Shift 限制形狀；Polygon 可用雙擊或 Enter 完成，並可用 Backspace 移除最後一個頂點、Esc 取消這次繪製。

## 2.6 視窗與對話框

New Window 會建立另一個繪圖視窗，而且每個視窗各自保有 Document 與操作紀錄。文字輸入、檔名輸入與覆寫確認都使用相同的 Element、Layout 與 Event 系統。當 Modal Dialog 開啟時，其他視窗會暫停接收使用者輸入，避免使用者同時操作背景視窗。若 Document 還有未儲存的修改，New、Load 與 Close 也會先顯示確認視窗。

# 3 系統架構與設計

## 3.1 整體架構

圖 2 整理了程式中幾個主要部分。Application 負責建立與保留各個 Window；PaintWindow 將操作介面、選單、快捷鍵與 Document 組合起來。使用者完成一次繪圖後，Canvas 會把結果交給 Document，Document 再修改 Scene。Renderer 只讀取 Scene 的內容，依物件種類選擇圖形或文字的繪製方式。

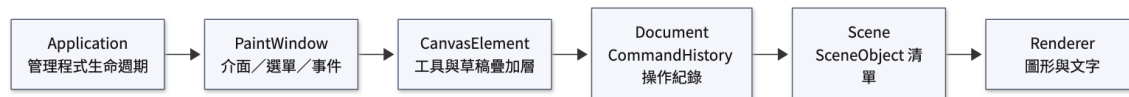


Figure 2: 繪圖程式的整體架構

## 3.2 Application 與 Window

FreeGLUT 本身可以建立多個視窗，但不會替 C++ 程式管理物件生命週期。我因此讓 Application 管理所有 Window，新的 Paint Window、Input Dialog、Confirm Dialog 與圖片輸出用視窗都從同一個入口建立。視窗收到關閉要求後先標記狀態，之後再由 idle callback 移除，避免 callback 還在執行時物件就被釋放。

另一個問題是 FreeGLUT 只接受 C-style callback，不能直接綁定某個 Window instance。程式建立一張以 GLUT window ID 為 key 的查找表；static callback 先取得目前視窗 ID，再找回真正要處理事件的 Window。程式碼 1 是這段轉接的核心，完整流程則畫在圖 3。多視窗與 Modal Dialog 的生命週期放在附錄 A。

Hierarchical Menu 也有類似限制：FreeGLUT 的 menu callback 只會傳回整數 ID。Menu class 因此替每個選項記錄 ID 與動作的對照，收到 callback 後先找回項目，再執行綁定的功能。這讓 PaintWindow 可以用一般 lambda 設定選單行為，不

必把所有功能塞進同一個 switch。

```

1 Window* Window::currentWindow() {
2     const int id = glutGetWindow();
3     auto it = windows.find(id);
4     if (it != windows.end()) {
5         return it->second;
6     }
7     return nullptr;
8 }

```

Listing 1: 由 GLUT window ID 找回 C++ Window instance



Figure 3: GLUT 回呼的路由流程

### 3.3 Document、Scene 與 Shape

Document 記錄檔名、Canvas 尺寸、Scene 與 Command History，是繪圖內容修改的入口。Scene 依照繪製順序儲存 SceneObject，其中 ShapeObject 包含 Point、Line、Rectangle、Ellipse、Path 與 Polygon，TextObject 則記錄文字位置、UTF-8 字串與文字樣式。

這樣分層之後，Scene 不需要知道 UI，也不會直接呼叫 OpenGL。工具完成時只產生一個 SceneObject，再由 Document 建立 Command 修改 Scene。拖曳中的 draft 留在 tool overlay，所以尚未完成的圖形不會先進入 Scene，也不會多產生一筆 Undo 紀錄。

### 3.4 Event 與 Input

滑鼠事件進入 Window 後，程式會先從 Element tree 中找出游標所在的最深層元件；若某個元件已取得 mouse capture，就直接把事件送到該元件。Listener 執行完後，Input Event 會沿著 parent chain 向上傳遞，最後到達 Window。元件如果已經處理完事件，可以呼叫 stopPropagation()，避免同一個輸入又被上層重複處理。圖 4 顯示這個過程。

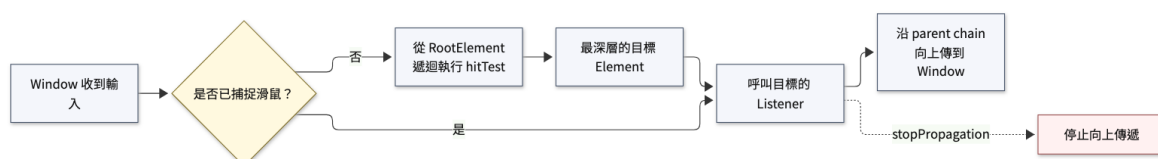


Figure 4: Event 目標選取與沿父節點向上傳遞的流程

每個輸入事件都會附帶當下的 **keyboard** 與 **mouse state**，因此工具在處理 **Motion** 時，可以同時判斷 **Shift** 是否按下以及 **MouseLeft** 是否仍在拖曳。**ShortcutManager** 若遇到多個可能的 **KeyChord**，會優先選擇按鍵數較多的組合，避免較短的快捷鍵先被觸發。

### 3.5 Command History 與 Undo/Redo

加入、插入、移除物件與清空畫布都會建立可以復原的 **Command**。新的 **Command** 執行後會進入 **undo stack**，同時清空 **redo stack**；**Undo** 會執行反向操作，並把同一筆紀錄移到 **redo stack**，**Redo** 則再次執行它。除了兩個 **stack** 之外，每筆紀錄還記錄操作前後的 **state ID**，讓程式可以判斷目前內容是否回到上次儲存的狀態。圖 5 整理了這個流程。

程式碼 2 節錄實際的 **stack** 移動方式。執行新操作時先呼叫 **Command**，再記錄前後狀態並清空 **redo stack**；**Undo** 則把最後一筆紀錄移到另一側。

```

1 void CommandHistory::execute(std::unique_ptr<IUndoableCommand> command) {
2     command->execute();
3     const StateId next = _nextState++;
4     _undoStack.push_back({std::move(command), _currentState, next});
5     _currentState = next;
6     _redoStack.clear();
7 }
8
9 void CommandHistory::undo() {
10     if (_undoStack.empty()) return;
11     auto item = std::move(_undoStack.back());
12     _undoStack.pop_back();
13     item.command->undo();
14     _currentState = item.stateBefore;
15     _redoStack.push_back(std::move(item));
16 }

```

Listing 2: Command 執行與 Undo stack 的更新

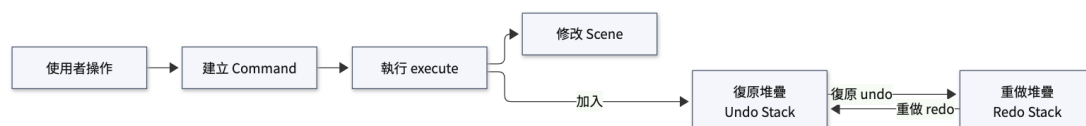


Figure 5: Command、復原與重做的資料流

### 3.6 Rendering Pipeline

Window 開始重繪時，Element tree 會按照各自的局部座標依序 render。輪到 Canvas 之後，CanvasRenderer 先畫 Grid，再按照 Scene 中的順序畫出物件，最後補上工具的 draft overlay。SceneRenderer 會判斷物件是 Shape 還是 Text，再交給對應的 Renderer 呼叫 OpenGL。整條 Rendering Pipeline 如圖 6 所示。



Figure 6: 目前的畫面渲染流程

## 4 繪圖與渲染實作

### 4.1 基本圖形

每種幾何圖形都能提供 Renderer 所需的頂點。Line 回傳兩端點，Rectangle 回傳四個角，Ellipse 以固定 64 個 segment 取樣，Path 與 Polygon 則使用工具收集的點。Point 是較簡單的特例，直接以 GL\_POINTS 和目前的 point size 繪製。Renderer 因此只需要處理完成後的 geometry，不用知道前面是單擊、拖曳還是多次點擊。

### 4.2 Interactive Drawing 與 Draft

繪圖工具不會在 Mouse Down 時立刻修改 Document，而是先建立一個 draft。接下來的 Motion 只改變 draft 的座標並要求重畫，所以使用者可以立即看到預覽。直到 Mouse Up、DoubleClick 或 Enter 表示操作完成，Canvas 才把新物件交給 Document 與 Command History。圖 7 是 DragTool 和 PolygonTool 共用的主要流程。

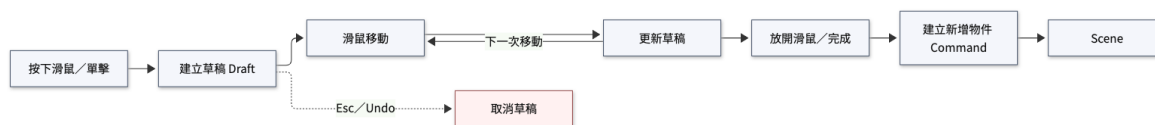


Figure 7: 繪圖工具與草稿的生命週期

切換工具時，目前工具會先處理尚未完成的 draft。若使用者在互動過程中按下 Undo，程式先取消 draft，而不是立刻復原 Scene 中上一個物件。這讓「取消這次尚未完成的動作」和「復原前一次已完成的動作」不會混在一起。



### 4.3 Stroke Rendering

Advanced mode 不直接依賴 OpenGL 的內建粗線，而是自行產生 triangle geometry。假設相鄰三點為  $A$ 、 $B$ 、 $C$ ，先計算兩段的單位方向向量

$$\vec{u} = \frac{B - A}{\|B - A\|}, \quad \vec{v} = \frac{C - B}{\|C - B\|}.$$

接著將方向向量旋轉  $90^\circ$  得到法向量  $\vec{n}_0$ 、 $\vec{n}_1$ ，並以半寬  $h = w/2$  建立左右兩側的 offset line，例如第一段的邊界會通過  $A \pm h\vec{n}_0$  與  $B \pm h\vec{n}_0$ 。程式求出兩組 offset line 的交點，再用二維 cross product 判斷轉向：

$$\vec{u} \times \vec{v} = u_x v_y - u_y v_x.$$

Canvas 的 Y 軸向下，因此程式中的結果大於 0 表示向右轉，小於 0 表示向左轉；這個符號接著用來選擇哪一側是外側。MITER 使用外側交點，BEVEL 用一個 triangle 截去尖角，ROUND 則補上一段 triangle fan arc。圖 8 比較三種結果。

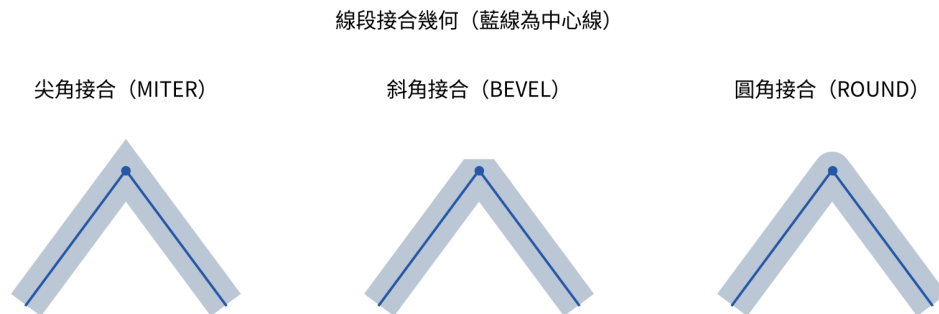


Figure 8: MITER、BEVEL 與 ROUND 接合方式比較；細線為中心線

開放路徑的端點只有一個方向。BUTT 直接停在原端點；SQUARE 沿線段方向再延伸  $h$ ；ROUND 則以端點為圓心補上半圓。三種結果如圖 9。這些端點不是交給 `glLineWidth()` 決定，而是由同一套 geometry 計算產生。

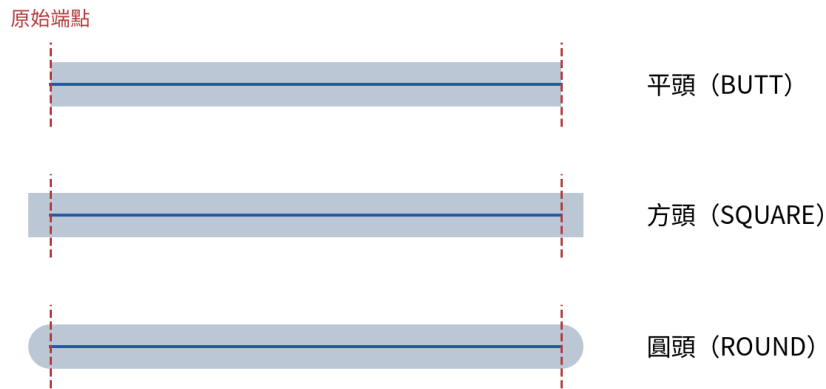


Figure 9: BUTT、SQUARE 與 ROUND 端點樣式比較；虛線為原始端點

當兩段線形成很尖的角度時，外側 offset line 的交點可能離  $B$  很遠。程式將這段距離記為 miter length  $m$ ，並與  $hL$  比較，其中  $L$  是 miter limit，目前預設為 4：

使用 MITER 的條件： $m \leq hL$ 。

若  $m > hL$ ，即使使用者選擇 MITER，程式仍改用 BEVEL，避免出現過長的尖刺。圖 10 顯示這項判斷。

程式碼 3 是 Renderer 中的實際判斷。只有選擇 MITER 且外側交點距離沒有超過限制時，才會保留兩側交點；其餘情況再依設定改畫 BEVEL 或 ROUND。

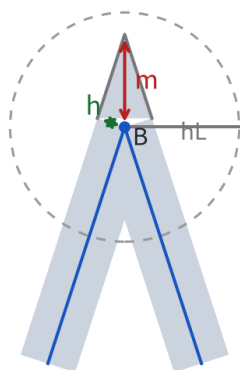
```

1 const auto turn = cross(u, v);
2 const bool turnRight = turn > 0.0f;
3
4 Point outerInter = turnRight ? leftInter : rightInter;
5 const GLfloat outerDistance = abs(outerInter - curr);
6 const bool validMiter = style.join == LineJoin::MITER &&
7                       outerDistance <= h * style.miterLimit;
8
9 if (validMiter) {
10     result.joinType = LineJoin::MITER;
11     result.inLeft = result.outLeft = leftInter;
12     result.inRight = result.outRight = rightInter;
13 }

```

Listing 3: Stroke Join 的 miter limit 判斷

頂點夾角 =  $36^\circ$   
 $m/h = 3.2 \leq L = 4$ ：使用尖角接合 (MITER)



頂點夾角 =  $18^\circ$   
 $m/h = 6.4 > L = 4$ ：改用斜角接合 (BEVEL)

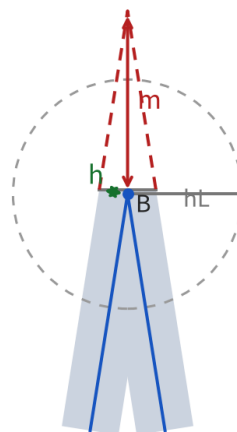


Figure 10: Miter limit 的實際轉角案例；灰底表示筆畫寬度，藍線為中心線

程式也處理平行、近  $180^\circ$  折返與過遠 inner intersection。若 offset line 無法穩定求交，Join 退回 BEVEL 或 ROUND；inner intersection 超過附近 segment 長度與半寬推導出的限制時，改用原 vertex，避免短筆畫出現異常 triangle。

## 4.4 Fill Rendering

OUTLINE 與 FILLED 模式直接使用 OpenGL primitive 和 `glPolygonMode()`。ADVANCED 模式則把填色與描邊分開：封閉圖形先填入 Fill color，再畫出自行計算的 Stroke geometry。若 Fill color 完全透明或頂點少於三個，就跳過填色。目前填色仍使用 `GL_POLYGON`，因此凹多邊形可能無法得到正確結果，這部分尚未加入 triangulation。

## 4.5 座標系統

Window 收到 reshape callback 後，會將 viewport 設成新的視窗大小，再以 `glOrtho(0, width, height, 0, -1, 1)` 建立 orthographic projection。這樣視窗左上角就是 (0,0)，X 向右增加、Y 向下增加，方向與 GLUT 傳入的滑鼠座標相同。工具不需要在每次滑鼠事件中翻轉 Y 軸，也比較容易直接用 pixel 理解線寬、點大小與游標位置。

事件剛進入 Window 時使用的是視窗座標。Canvas 收到事件後，`windowToLocal()` 會沿著 parent chain 逐層扣除每個 Element 的 bounds 左上角，得到以 Canvas 左上角為原點的局部座標；繪圖工具、SceneObject 與 Grid 都使用這套 Canvas 座標。反方向的 `localToWindow()` 則逐層加回位移。由於目前 Element 沒有旋轉或縮

放，這兩個轉換只需要處理平移。

狀態列顯示的游標位置就是轉換後的 Canvas 座標，Canvas 尺寸也使用相同單位。這讓畫面上看到的位置、工具收到的座標與 SceneObject 儲存的頂點一致；只有讀寫 framebuffer 時需要另外處理 OpenGL 的左下角原點，相關內容放在第 6 節。

## 4.6 Grid

Grid 的用途是提供位置與距離的視覺參考，本身不會加入 Scene，也不會改變物件頂點，更沒有吸附功能。Lines 模式每隔 25 px 畫一條水平線和垂直線；Dots 模式改在相同交點畫 2 px 的點；None 則不畫格線。使用者可以從右鍵選單選擇模式，也可以按 G 依序切換。

CanvasRenderer 先畫 Grid，再畫已提交的 SceneObject，最後才畫工具的 draft。因此格線只留在背景，不會蓋住已完成的物件或拖曳中的預覽。圖 11 左側用一組座標示範 windowToLocal() 的結果，右側比較三種 Grid 模式與實際繪製順序。

Canvas 座標轉換與 Grid 模式

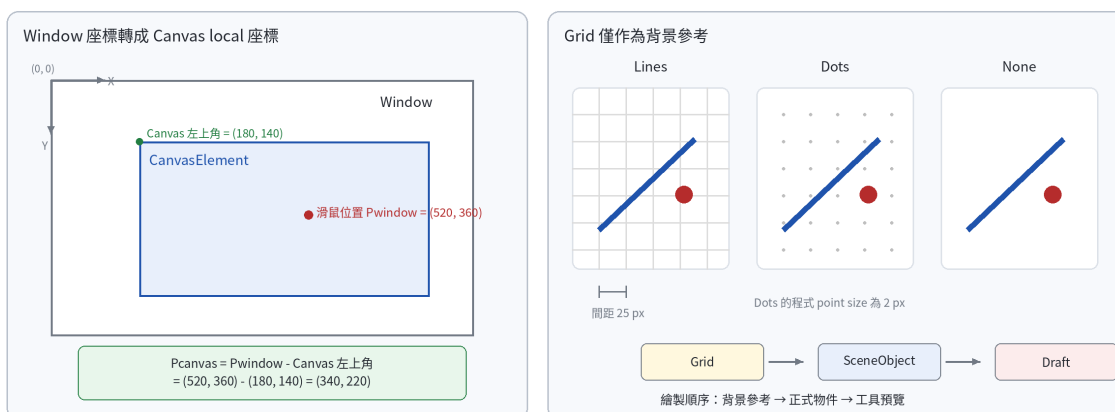


Figure 11: Canvas local 座標、Grid 模式與繪製順序

## 5 文字功能

### 5.1 FreeGLUT 字型

文字物件可以使用 FreeGLUT 提供的 Bitmap 與 Stroke 字型。Bitmap Renderer 逐字設定 raster position，再呼叫 glutBitmapCharacter()；由於不同字型的原點位置不一樣，程式也會調整 baseline。Stroke 字型原本使用 Y 軸向上的座標，繪製前會先翻轉 Y 軸，才能和 Canvas 的座標方向一致。這些內建字型主要用來顯示 ASCII。

## 5.2 Text Object 與文字輸入

選擇 Text tool 並點擊 Canvas 後，程式會開啟 Modal Input Dialog。只有在使用者確認文字後，才會建立 TextObject 並加入 Scene；如果取消輸入，Scene 不會留下空的文字物件。Dialog 只負責收集字串，實際修改 Document 的工作仍由 PaintWindow 完成。

輸入框內部以 Unicode code point 為編輯單位，對外仍使用 UTF-8 字串。這樣 Backspace、Delete 與左右方向鍵一次會移動或刪除一個字元，不會切斷中文字的 UTF-8 bytes。輸入框也支援 Home、End、255 個 code point 的長度上限，以及每 500 ms 切換一次的 caret。FreeGLUT 的一般 keyboard callback 只能直接提供單一 byte，因此程式另外加入反引號加四位十六進位數字的輸入方式，例如 ``4f60` 會輸入「你」。這不是完整的輸入法，但足以測試中文字元與 Glyph rendering。

圖 12 以「A 你 B」為例，對照畫面上的三個字元、各自的 Unicode code point 與 UTF-8 bytes。Caret index 記錄的是字元之間的位置，因此刪除「你」時會移除一個 code point，而不是只刪除其中一個 byte。

UTF-8 bytes、code point 與 caret 的對應

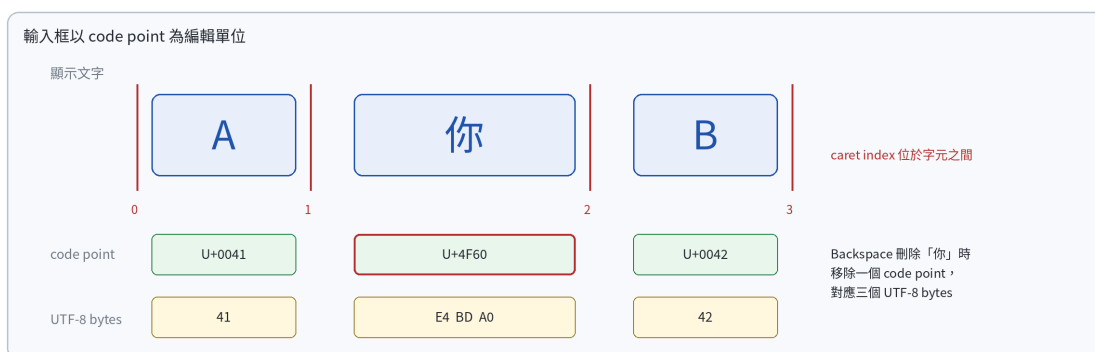


Figure 12: UTF-8 bytes、code point 與 caret index 的對應

電腦圖學

Figure 13: GFNT 在程式輸出中繪製「電腦圖學」的實際結果

圖 13 是從程式實際輸出的圖片裁切而成，可確認四個中文字都能找到對應 Glyph，

字元順序與基線也符合輸入內容。輸入過程中的 caret 與 code point 關係則由圖 12 說明。

### 5.3 UTF-8 與中文字元支援

我另外實作 UTF-8 decoder，支援 1–4 byte sequence，並排除 overlong encoding、超過 U+10FFFF 的值與 UTF-16 surrogate。遇到不合法的輸入時，以 U+FFFD 取代。輸入框會先把 UTF-8 轉成 code point 進行編輯，顯示時再轉回 UTF-8；TextRenderer 也使用相同的 decoder 逐字取得 code point。

### 5.4 自訂中文字型支援

為了顯示 FreeGLUT 內建字型沒有涵蓋的中文字，我設計了 GFNT bitmap font 格式。程式啟動時會載入 GFNT binary resource，文字繪製時先從 UTF-8 取出 code point，再查找 Glyph 的尺寸、bearing、advance 與 bitmap 位置。Renderer 依目前文字顏色組成 RGBA pixels，最後交給 OpenGL 繪製。圖 14 顯示文字從 UTF-8 到畫面的步驟，格式與載入檢查則放在附錄 B。

Glyph Table 依 code point 排序，所以實際查找使用 `std::lower_bound`，如程式碼 4。找不到原字元時，程式再嘗試 U+FFFD 與問號；完整 fallback 流程與檔案格式放在附錄 B。

```

1 const gfnt::GfntGlyphV1* GfntFont::findGlyph(char32_t codepoint) const {
2     const auto value = static_cast<std::uint32_t>(codepoint);
3     const auto it = std::lower_bound(
4         _glyphs.begin(), _glyphs.end(), value,
5         [](const gfnt::GfntGlyphV1& glyph, std::uint32_t searched) {
6             return glyph.codepoint < searched;
7         });
8
9     if (it == _glyphs.end() || it->codepoint != value) return nullptr;
10    return &*it;
11 }

```

Listing 4: 依 code point 查找 GFNT Glyph



Figure 14: GFNT 執行期文字渲染流程

## 6 Framebuffer 與輸出

### 6.1 Color Buffer

ColorBuffer 以 RGBA 格式儲存 framebuffer，每個 channel 使用 1 byte。擷取時，程式先把 UI 的左上角座標換成 OpenGL 的左下角座標，再從 single-buffer 的 front buffer 呼叫 `glReadPixels()`。還原畫面會暫時關閉 `blending`、`depth`、`stencil`、`scissor` 與 `alpha test`，接著用 `glDrawPixels()` 寫回像素。函式結束前會還原 `PACK/UNPACK alignment` 與其他 OpenGL state，避免影響後續繪圖。

圖 15 顯示同一個擷取區域在兩套座標中的位置。UI 以左上角為原點，但 `glReadPixels()` 的位置必須從 framebuffer 左下角計算，因此程式使用視窗高度、區域上緣與區域高度求出 `bottom`。讀回的 `pixels` 仍由下往上排列，座標轉換本身不會翻轉資料。

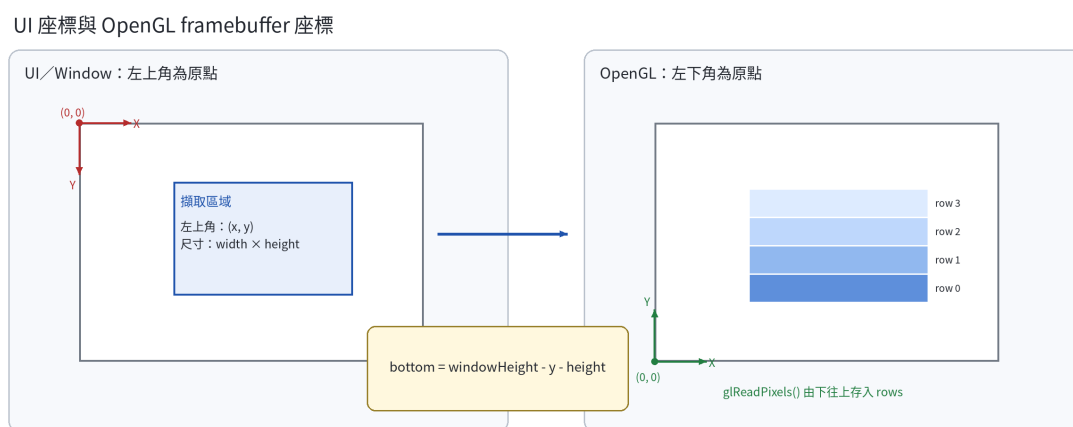


Figure 15: UI 與 OpenGL framebuffer 的座標及 pixel row 順序

### 6.2 Display Restore

Window 會分別記錄「內容需要重畫」與「畫面需要重新擷取」兩種狀態。內容改變或 `cache` 尺寸不符時，`display callback` 先重畫整棵 `Element tree`；等畫面完成而且沒有新的修改後，`timer` 才把 `framebuffer` 存入 `ColorBuffer`。下一次若只是視窗重新曝光，就能直接還原像素，不必重新走一次所有 `Renderer`。狀態轉換如圖 16。

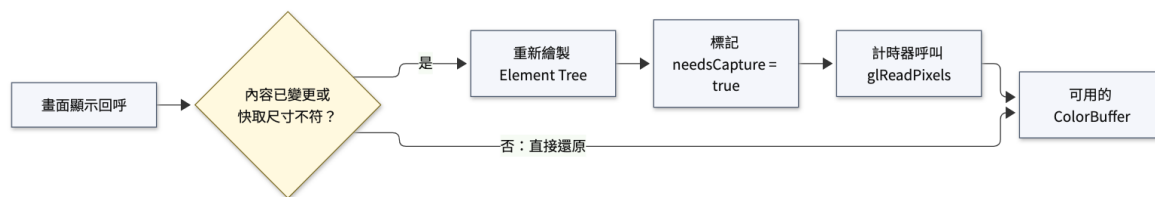


Figure 16: ColorBuffer 快取與畫面還原流程

### 6.3 圖片輸出

輸出圖片時，程式會建立一個與 Canvas 相同尺寸的 GraphicsRenderWindow，關閉 Grid，並重用平常的 Canvas Renderer 重新畫出 Scene。呼叫 `glFinish()` 確認繪圖完成後，再讀取 framebuffer，依 PPM P6 格式由上到下寫出 RGB bytes，alpha channel 不輸出。這種做法不會把主視窗的狀態列一起存進圖片。PNG 與 JPEG 目前沒有實作。

由於 ColorBuffer 的第 0 列位於畫面下方，PpmExporter 會從 `height - 1` 反向走訪到第 0 列。圖 17 以小型 pixel matrix 表示這個轉換，也標出 P6 header 與忽略 alpha channel 的位置。

ColorBuffer 到 PPM P6 的輸出順序

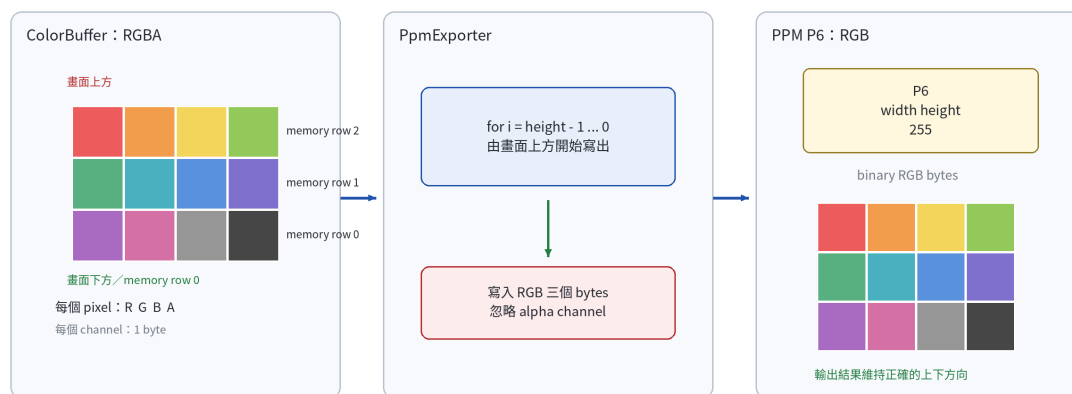


Figure 17: ColorBuffer 轉成 PPM P6 時的列順序與 channel 配置





Figure 18: 程式實際輸出的 PPM 圖片；畫面只保留 Scene 內容

圖 18 是由程式產生的實際檔案，輸出尺寸為  $800 \times 578$ 。圖中的自由曲線與「電腦圖學」文字方向正確，而且沒有主視窗的 Grid 與狀態列，表示輸出視窗只重畫 Scene 內容。

## 7 特殊貢獻摘要

除了題目要求的基本 drawing panel，我另外完成了幾項和程式結構或繪圖效果有關的功能。前面章節已經說明過實作細節，因此這裡只整理各項功能的目的與對應章節。

Table 3: 特殊功能摘要

| 特殊功能                   | 簡要說明                                                      | 詳細章節    |
|------------------------|-----------------------------------------------------------|---------|
| <b>Undo / Redo</b>     | 以 Command History 保留操作，並用 state ID 判斷 Document 是否回到已儲存狀態。 | 第 3.5 節 |
| <b>Multi-window</b>    | 以 Window ID 找回 C++ instance，並讓 Modal Dialog 暫停背景視窗輸入。     | 附錄 A    |
| <b>Advanced Stroke</b> | 自行計算 Join、Cap 與 Miter Limit 所需的 triangle geometry。        | 第 4.3 節 |
| <b>自製 GUI</b>          | 實作 Element tree、Layout、Event bubbling 與 mouse capture。    | 附錄 A    |
| <b>GFNT</b>            | 自行處理 UTF-8、Glyph lookup 與 bitmap font rendering。          | 附錄 B    |

這些功能都直接服務於繪圖程式：Command History 讓操作可以復原，GUI framework 支撐多視窗與 Dialog，Stroke geometry 改善粗線外觀，GFNT 則補上中文顯示。它們沒有改變作業的核心目標，但讓我可以實際處理較完整的互動與 rendering 問題。

## 8 成果展示與作業需求驗證

### 8.1 繪圖成果

圖 19 集中呈現 Point、Pencil、Line、Rectangle、Ellipse、Polygon 與 Text 的操作結果，並比較不同 Fill、Stroke、Join、Cap 與 Grid 設定。

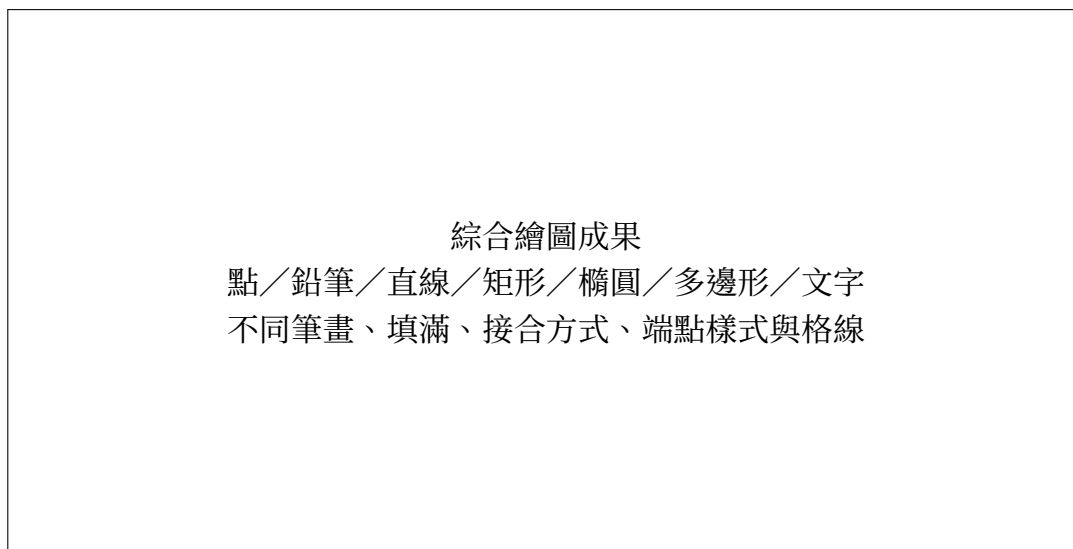


Figure 19: 基本工具與進階筆畫樣式的繪圖成果

## 8.2 進階功能成果

圖 13 顯示文字輸入，圖 18 對照 Canvas 與 PPM 輸出。圖 20 則呈現兩個 PaintWindow 和 Confirm Dialog，用來觀察不同視窗的 Document 是否彼此獨立，以及 Dialog 開啟時背景視窗的輸入狀態。



Figure 20: 多視窗與確認對話框

## 8.3 Requirement Checklist

表 4 依 README 與實際程式碼標示完成狀態。介面中存在但 callback 為空，或 file command 存在但內容未序列化者，都不列為完成。

Table 4: 作業要求與實作狀態

| Requirement                            | Status | Implementation                                                                   |
|----------------------------------------|--------|----------------------------------------------------------------------------------|
| Interactive 2D drawing panel           | 已完成    | CanvasElement、tool draft、Scene 與 Renderer。                                       |
| Hierarchical menus                     | 已完成    | FreeGLUT Menu 封裝與 File/Edit/Tools/Stroke/Fill/Point/Text/Grid 子選單。               |
| Keyboard events                        | 已完成    | KeyDown/KeyUp/TextInput、InputState 與 ShortcutManager。                            |
| Mouse / motion events                  | 已完成    | MouseDown/Up/Move、Click/DoubleClick、hit test 與 mouse capture。                    |
| Window events                          | 已完成    | Close、Reshape、Visibility、Entry callback 轉為 Window Event。                         |
| Color                                  | 部分完成   | Stroke/Fill/Text 預設色可選；Custom color callback 尚未完成。                               |
| Line width / point size                | 已完成    | 選單與中括號快捷鍵可調整。                                                                    |
| Fill mode                              | 已完成    | OUTLINE、FILLED、ADVANCED；Advanced 分開處理 Fill 與 Stroke。                             |
| Mouse / motion drawing                 | 已完成    | DragTool、PencilTool 與 PolygonTool。                                               |
| Color buffer save / restore            | 已完成    | ColorBuffer 使用 glReadPixels / glDrawPixels 做視窗 cache。                            |
| Save color buffer                      | 已完成    | PPM P6 export 輸出重新 render 後的 Canvas framebuffer。                                 |
| Text input                             | 已完成    | Modal InputDialog、InputElement、TextObject 與三種字型 renderer。                        |
| Grid                                   | 已完成    | 25 px line grid、dot grid 與 none。                                                 |
| Grid-line / scales / positions / sizes | 部分完成   | 已有 line/dot Grid，Status Bar 會顯示游標位置與 Canvas 尺寸；程式中沒有獨立的 scale/ruler。             |
| Undo / Redo                            | 已完成    | CommandHistory 與 Add/Insert/Remove/Clear commands。                               |
| Multi-window                           | 已完成    | Application 管理視窗，Window registry 負責 callback routing；Modal Dialog 開啟時背景視窗停止接收輸入。 |
| Save / Load document                   | 部分完成   | .gpt 可寫 header/END 並有 UI/Command；Scene serialization/deserialization 尚未完成。       |

| Requirement       | Status | Implementation                                            |
|-------------------|--------|-----------------------------------------------------------|
| Selection tool    | 尚未完成   | Tools/Select 選單 callback 為空。                              |
| PNG / JPEG export | 尚未完成   | 僅支援 binary PPM。                                           |
| Fancy ideas       | 已完成    | 自製 GUI、advanced stroke、GFNT、UTF-8、多視窗與 framebuffer cache。 |

## 9 開發問題與解決方法

### 9.1 GLUT Callback 與多視窗狀態

剛開始實作時，最直接的寫法是在 GLUT callback 裡讀取滑鼠位置、修改目前圖形，再呼叫 OpenGL 重畫。單一視窗與少量工具時還能維持，但加入對話框和多視窗後就出現問題：FreeGLUT 的 callback 不會附帶 C++ object pointer，同一組 static callback 也可能由不同視窗觸發。如果只靠全域變數記錄「目前視窗」，輸入很容易被送到錯誤的 Document。

我的處理方式是保留 GLUT window ID 到 Window\* 的查找表。每個 static callback 先用 glutGetWindow() 找出來源，再轉交給對應的 Window。FreeGLUT menu 只會回傳整數 ID，因此 Menu 也建立自己的 ID 對照。視窗關閉時先移除這些 ID；timer callback 執行前則再次確認目標是否仍存在。這層 routing 讓 GLUT 保持在最外圍，後面的 Event、Element 與 Document 不需要知道 callback 是 static function。

### 9.2 從原始輸入整理出可用的 Event

GLUT 提供的是 MouseDown、MouseUp、Motion 和鍵盤 callback，但繪圖工具實際需要 Click、DoubleClick、drag、focus 與 mouse capture。另一個限制是，游標拖出 Canvas 後，單靠 hit test 會把後續 Motion 送到別的 Element，最後的 MouseUp 也可能回不到原工具，留下尚未結束的 draft。

因此 Window 先記錄 keyboard、mouse、focus、hover 與 click history，再把 callback 資料轉成 Event。按下與放開的位置相差不超過 4 px 才算 Click；兩次 Click 的時間不超過 300 ms 且位置接近時才產生 DoubleClick。Canvas 在 MouseDown 後取得 mouse capture，使後續 MouseMove 與 MouseUp 固定回到同一個元件；Element 從 tree 移除時，Window 也會清除落在該子樹中的 focus、hover 與 capture pointer。這些機制完成後，各個 Tool 才能用一致的事件流程處理互動。

### 9.3 Layout、座標與 Framebuffer 的方向差異

UI、滑鼠與 Scene 都採左上角原點、Y 向下，但 OpenGL framebuffer 的 pixel row 從左下角開始。一般繪圖可以用 orthographic projection 對齊 GLUT 座標，Element 再透過 parent bounds 把視窗座標轉成局部座標；讀回 ColorBuffer 或輸出 PPM 時，卻仍要處理 framebuffer 的 row order。若直接依讀回順序寫檔，圖片就會上下顛倒。

我把 Element 的位置計算分成 measure 和 arrange，並要求 render 與 hit test 前先完成需要的 Layout。ColorBuffer 則單獨儲存 framebuffer 內容，輸出 PPM 時反向走訪 pixel rows。Timer 需要擷取畫面時，會先切換到指定的 GLUT window；完成後只有在原本的 window ID 仍有效時才切回。這樣多視窗、命中測試和圖片輸出可以共用座標定義，又不會忽略 OpenGL pixel I/O 的方向。

### 9.4 讓操作可以預覽，也可以 Undo / Redo

拖曳矩形或直線時需要即時預覽，但若每次 Motion 都直接加入 Scene，Undo history 會累積大量中間狀態；Polygon 又要跨越多次 Click 才能完成，不能套用單純的 MouseDown/MouseUp 流程。我最後把尚未完成的圖形留在 Tool 的 draft 中，Renderer 把它當成 overlay 畫在 Scene 後面。只有 Tool 回傳完成的物件時，Document 才建立 Command 並修改 Scene。

這項分工也解決了取消操作的問題。繪圖中按 Undo 時，Canvas 先取消 draft；沒有 draft 時才復原上一個 Command。新的 Command 執行後會清除 redo stack，而 Document 以 state ID 判斷內容是否回到上次儲存的狀態。如此一來，預覽、正式資料與操作紀錄各自有清楚的生命週期。

### 9.5 粗線轉角的幾何例外

自行產生粗線 triangle 時，直線本身不難，真正麻煩的是轉角。角度很尖時，兩條外側 offset line 的交點會離頂點很遠；接近 180 度折返或 segment 太短時，內側交點也可能不穩定。如果直接接受所有交點，畫面會出現長尖刺或互相交錯的 triangle。

程式先移除相鄰的重複點，再用 cross product 與 dot product 區分同向、一般轉彎和折返。MITER 只有在 miter length 不超過半線寬乘上 miter limit 時才使用，否則退回 BEVEL；內側交點也依附近 segment 長度設定距離上限。求交失敗時，BEVEL 與 ROUND 各自有不依賴遠端交點的替代畫法。這些條件不是為了增加選項，而是避免使用者快速畫出短折線時產生明顯破圖。

## 9.6 文字輸入與 GFNT 資料檢查

FreeGLUT 的一般 keyboard callback 一次提供一個 byte，無法直接當成完整的 UTF-8 字元。若輸入框又用 byte index 移動 caret，Backspace 可能只刪掉中文字的一部分。我的做法是把編輯中的文字轉成 code point vector，caret index 放在字元之間，顯示時才重新編碼成 UTF-8。Glyph 寬度則由目前 FontStyle 計算，讓 caret 能跟著實際顯示位置移動。

GFNT Loader 也不能直接假設檔案內容正確。它先逐欄讀取 little-endian 數值，確認 Header 和 Glyph Table 沒有超出檔案，再檢查 code point 排序、Unicode 合法範圍與每張 bitmap 的邊界。完成載入時的一次性檢查後，render loop 才使用 binary search 查找 Glyph。這讓文字繪製的主要路徑保持簡單，也能在字型檔損壞時提早停止，而不是等到 OpenGL 讀取錯誤位置才出現問題。

## 10 已知限制與未來改進

目前最直接的限制是 .gpt 檔案只會寫出檔頭與結尾標記，讀取時也還沒有解析 Scene，因此 Save/Load 不能真正儲存繪圖內容。下一步需要先定義各種 SceneObject 與 Style 的序列化格式，再測試存檔後重新開啟是否能還原相同內容。現階段若要保留畫面，應使用 PPM export。

Select 與 Custom color 已經出現在 menu 中，但 callback 仍是空的。若繼續擴充，我會先完成 selection model 與物件 hit testing，再讓 Move 和 Delete 也透過 Command 執行。Renderer 方面，GL\_POLYGON 對凹多邊形不可靠，之後可加入 triangulation；Ellipse 固定使用 64 個 segments，也可以依半徑調整。圖片輸出目前只有 PPM，尚未支援 PNG 或 JPEG。Grid、游標位置與 Canvas 尺寸已經能顯示，但程式沒有額外的 scale/ruler。

## 11 心得

一開始我把這個作業理解成在 GLUT callback 裡切換工具，再直接呼叫 OpenGL 畫圖。這樣做前幾個功能很快，但加入多邊形、文字、Undo/Redo、對話框與多視窗後，callback 會同時負責輸入狀態、Document 修改和 rendering，變得很難判斷一個事件到底改了什麼。因此我逐步拆出 Window、Event、Document、Tool、Command 與 Renderer，讓每一層只處理自己掌握的狀態。

這次最花時間的部分不是基本圖形，而是功能交界處。例如拖曳離開 Canvas 後 MouseUp 還要送回原工具；粗線轉角要處理接近平行與折返；中文字不能用 byte index 移動 caret；多視窗 timer 也不能假設目前 OpenGL context 永遠正確。把這些情況做完後，我比較能理解 GUI framework 為什麼需要 focus、capture、layout 與

event routing，而不是只有幾個 callback。

專案範圍後來確實比 drawing panel 大。如果重做一次，我會先固定最小需求：Scene、幾種基本工具、Command 與一條 Rendering Pipeline，等檔案格式真的能存檔和還原後，再加入自製 GUI 與 GFNT。這也讓我發現，UI 上已經出現功能入口，不代表功能本身就完成了。之後如果再做類似專案，我會更早把資料層、操作流程與 UI 一起完成，而不是先把選單項目做出來。

## A 自製 GUI Framework

### A.1 設計動機與範圍

這次作業沒有使用第三方 GUI framework，而是依照 drawing panel 的需要自行實作。我的目標不是做一套通用的桌面 toolkit，而是處理這個程式實際用到的多視窗、基本 control、Layout、focus、mouse capture、Event bubbling 與 OpenGL rendering。圖 21 顯示主要 class 之間的關係。

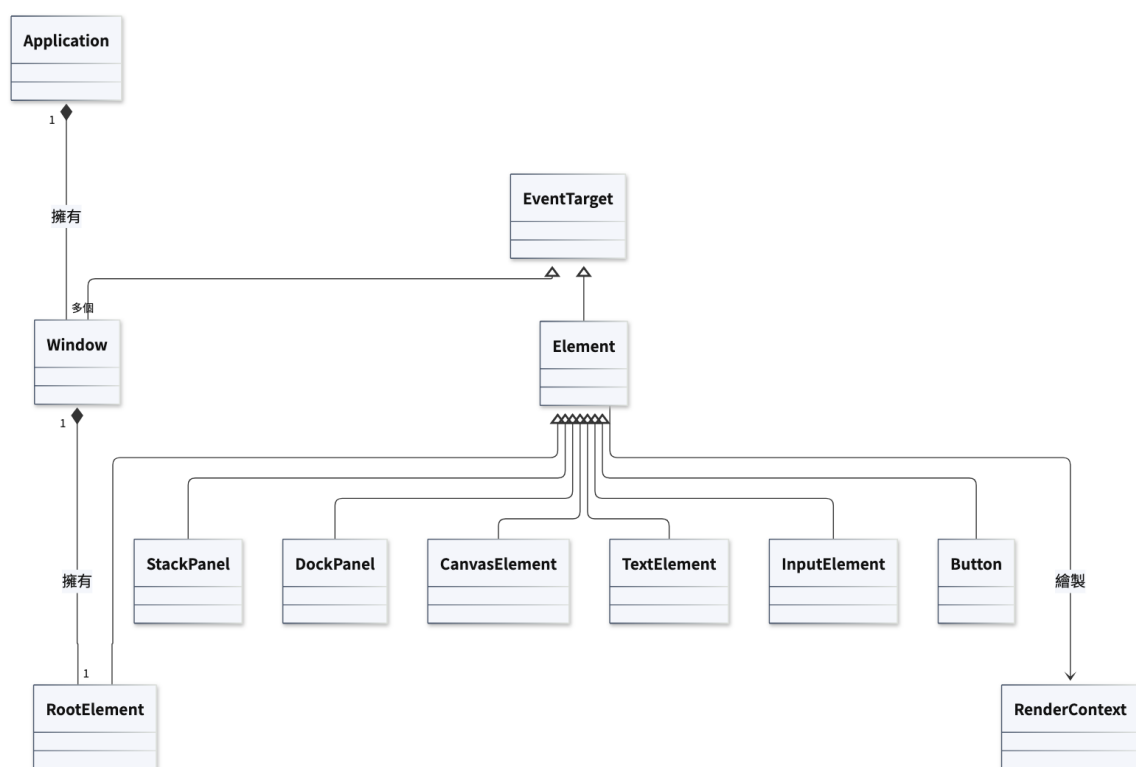


Figure 21: GUI 架構的主要類別關係



## A.2 Application 與 Window

Application 建立並管理所有 Window。Window 內部記錄 GLUT window ID、大小、標題、RootElement、ColorBuffer 與互動狀態。每個 Window 都註冊相同的 static callbacks，再透過圖 3 的 ID 查找方式找回正確 instance。PaintWindow、InputDialogWindow、ConfirmWindow 與 GraphicsRenderWindow 都建立在這一層之上。

## A.3 Element Tree 與 Layout System

每個 Element 記錄 parent、children、margin、padding、alignment 與 bounds。Layout 分成 measure 和 arrange 兩個階段：measure 先由下而上詢問每個元件需要多少空間，arrange 再由上而下分配實際位置。RootElement 只放一個主要內容；StackPanel 沿水平或垂直方向排列，並依 grow 權重分配剩餘空間；DockPanel 依加入順序從四邊扣除空間，最後一個 child 取得剩下區域。圖 22 是 PaintWindow 實際使用的 UI tree，圖 23 則整理從屬性變更到重新排版與繪製的流程。

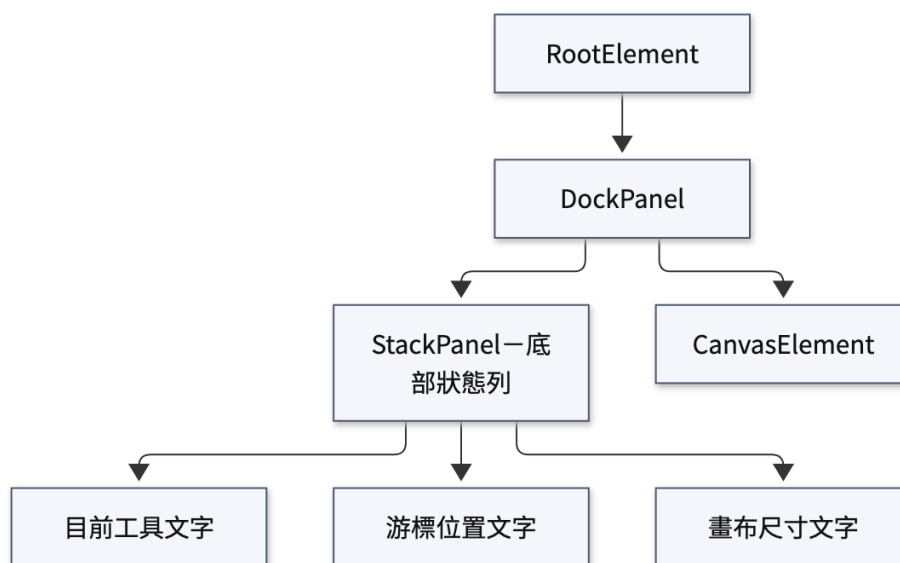


Figure 22: PaintWindow 的 Element 與 Layout 樹狀結構

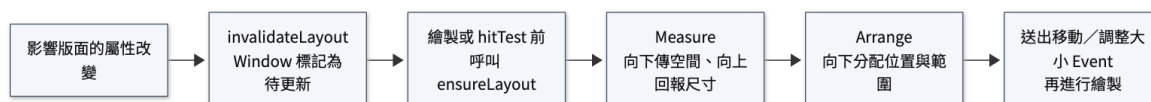


Figure 23: Element Tree 的測量、配置與重新繪製流程

當 preferred size、margin、padding 或 alignment 改變時，Element 會呼叫 invalidateLayout()，再由所屬 Window 把 Layout 標成需要更新。下一次 render

或 hit test 前，Window 先呼叫 `ensureLayout()`：measure 從 `RootElement` 往子節點傳入可用空間，子節點算完內容需求後再回報 `desired size`；`arrange` 則從視窗範圍開始，逐層分配實際 slot 與 `bounds`。這樣滑鼠命中測試使用的範圍會和畫面上實際繪製的位置一致。

## A.4 EventTarget、Bubbling 與 stopPropagation

`EventTarget` 依 `Event` 型別記錄 listener。事件開始傳遞時，`target` 會保持為最初接收的元件，`current target` 則隨著 bubbling 一層一層改變。`Element` 會把事件傳給 `parent`，`RootElement` 再往上交給 `Window`。輸入框或快捷鍵處理完按鍵後，可以呼叫 `stopPropagation()`，例如避免同一個 `Enter` 同時提交輸入框，又觸發上層 `Window` 的操作。傳遞流程見圖 4。

## A.5 Mouse Capture 與 Input State

`Window` 會記錄目前取得 `focus`、`hover` 與 `mouse capture` 的 `Element`。`MouseDown` 後 `Canvas` 取得 `capture`，所以 `Motion` 與 `MouseUp` 仍會送回 `Canvas`；`Hover` 不受 `capture` 影響，繼續依游標實際位置更新。鍵盤狀態由視窗共用，滑鼠狀態則由各 `Window` 記錄。`Modal Dialog` 開啟或滑鼠進出視窗時，程式會清理相關狀態，避免按鍵或滑鼠按鈕一直停在按下狀態。

## A.6 Multi-window Lifecycle

關閉 callback 不會立刻刪除 C++ object，而是先移除 `GLUT ID`，接著送出 `WindowCloseEvent`。`PaintWindow` 可以在這時檢查未儲存的修改並顯示 `Confirm Dialog`；使用者確定關閉後，`Window` 才會標記為可移除，最後由 `Application` 的 `idle callback` 釋放物件。`Timer` 每次執行前也會先確認視窗是否仍存在，避免 callback 執行到一半時物件已被銷毀。

# B GFNT 自訂點陣字型格式

## B.1 設計動機

`FreeGLUT` 內建 `Bitmap/Stroke` 字型不足以涵蓋中文，因此程式使用簡單的 `GFNT` `binary resource`。格式採用固定大小的 `header` 與 `glyph entry`，載入時可以先檢查檔案範圍，之後再依排序過的 `Unicode code point` 做 `binary search`。正文的圖 14 已整理文字繪製的主流程；本附錄再分別說明檔案結構、`Loader` 檢查與找不到字元時的 `fallback`。

## B.2 Binary Layout

格式使用 1-byte packing，並由 C++ 的 static assertion 確認 GfntHeaderV1 為 16 bytes、GfntGlyphV1 為 18 bytes。檔案中的整數欄位一律使用 little-endian。若 Glyph 數量為  $N$ ，Bitmap Data 的檔案 offset 就是  $16 + 18N$ 。圖 24 與表 5 都依照目前程式中的 struct 欄位與大小整理。

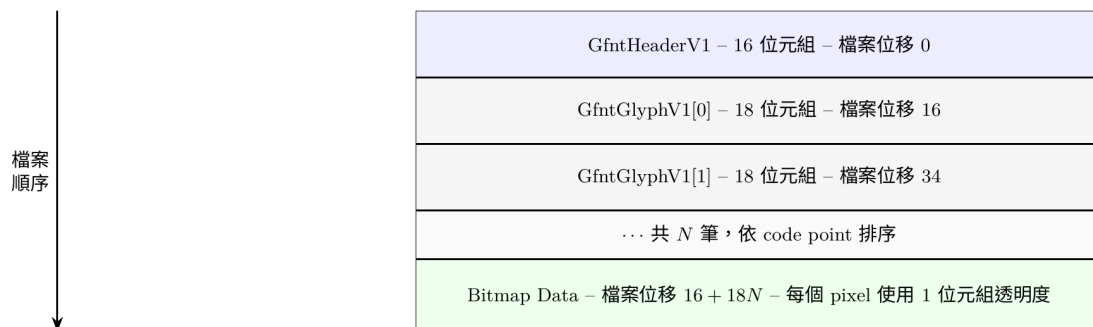


Figure 24: GFNT v1 二進位檔案配置

Table 5: GFNT v1 欄位、相對 offset 與大小

| Field                              | Offset | Size | Meaning                        |
|------------------------------------|--------|------|--------------------------------|
| <b>GfntHeaderV1 (16 bytes)</b>     |        |      |                                |
| magic                              | 0      | 4    | ASCII GFNT                     |
| version                            | 4      | 2    | 格式版本，目前為 1                     |
| pixelSize                          | 6      | 2    | nominal pixel size，不可為 0       |
| ascender                           | 8      | 2    | baseline 上方高度，int16_t          |
| descender                          | 10     | 2    | baseline 下方高度，int16_t          |
| glyphCount                         | 12     | 4    | glyph table entry 數量           |
| <b>GfntGlyphV1 (18 bytes each)</b> |        |      |                                |
| codepoint                          | 0      | 4    | Unicode codepoint              |
| width                              | 4      | 2    | bitmap 寬度                      |
| height                             | 6      | 2    | bitmap 高度                      |
| bearingX                           | 8      | 2    | pen 到 bitmap 左側的水平位移           |
| bearingY                           | 10     | 2    | baseline 到 bitmap 頂端的垂直位移      |
| advanceX                           | 12     | 2    | 下一個 pen X 增量                   |
| bitmapOffset                       | 14     | 4    | 相對 Bitmap Data 開頭的 byte offset |

### B.3 Loader 與驗證

Loader 先取得檔案大小，確認至少能容納 16-byte header，再逐欄讀取不同寬度的整數。接著檢查 magic、version、pixel size、ascender、descender，以及  $16 + 18N$  是否超出檔案。Bitmap Data 讀完後，程式還會逐一檢查 Glyph：code point 必須按照遞增順序排列，不得超過 U+10FFFF 或落在 surrogate 範圍，而且流程如圖 25 所示。

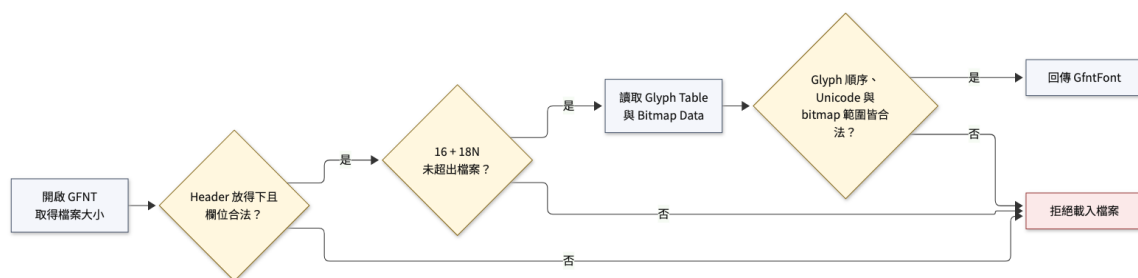


Figure 25: GFNT Loader 的讀取與驗證順序

$$o + w \times h \leq \text{Bitmap Data size.}$$

其中  $o$ 、 $w$ 、 $h$  分別對應 bitmapOffset、width 與 height。程式實際以減法形式比較，避免加法 overflow。這些檢查都在載入字型時完成，render loop 不需要每畫一個字就重新檢查。

### B.4 UTF-8 Decoder 與 Glyph Lookup

UTF-8 decoder 會檢查 sequence 長度、continuation bytes、最小合法值、最大 code point 與 surrogate，遇到錯誤時消耗一個 byte 並回傳 U+FFFD。Glyph Table 在載入時已確認順序，因此查找時使用 `std::lower_bound`；若找不到原字元，再依序嘗試 U+FFFD 與問號。圖 26 對應 `findGlyphOrFallback()` 的實際查找順序；三次都找不到時，Renderer 以 header 的 pixel size 推進 pen，而不繪製 bitmap。

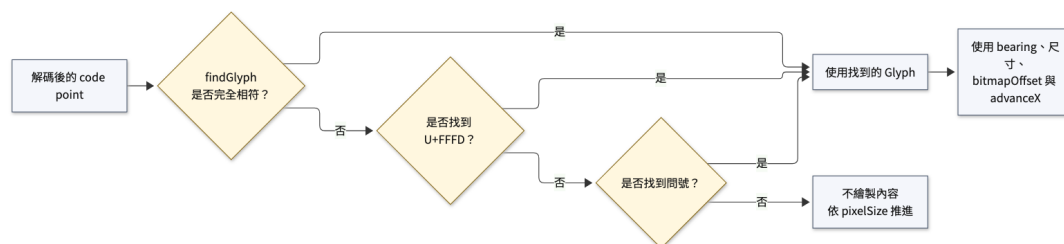


Figure 26: GFNT Glyph 查找與替代字元使用順序

## B.5 Bitmap Rendering

TextRenderer 以 header 的 ascender/descender 計算 baseline 與 line height。每個 glyph 的左上角為

$$x = \text{penX} + b_x, \quad y = \text{baselineY} - b_y.$$

其中  $b_x$  與  $b_y$  是 Glyph 的 horizontal bearing 與 vertical bearing。Bitmap Data 每個 pixel 儲存 alpha；Renderer 讀取目前的 OpenGL color，組成 RGBA pixels，再開啟一般 alpha blending。因為 Glyph rows 由上到下儲存，繪製時使用負的 Y pixel zoom。完成一個 Glyph 後，penX 再增加 advanceX。整個執行流程已在圖 14 表示。

圖 27 使用實際的 Cubic-11 與 Unifont 字型繪製「你」(U+4F60)。Cubic-11 雖然名稱包含 11，但本專案轉換時使用的 pixelSize 為 12，因此產生的「你」字 bitmap 為  $13 \times 11$  pixels，高度是 11 pixels。左半部依 Converter 產生 GFNT 欄位的方式，標出基線、bearing、bitmap 範圍與 advance；右半部則將 Unifont 的  $16 \times 16$  alpha bitmap 逐格畫出，並顯示 Renderer 如何把目前顏色乘上 Glyph alpha 形成 RGBA pixels。圖中的第一列對應 GFNT Bitmap Data 的起始位置，負的 Y pixel zoom 會讓 glDrawPixels() 由 Glyph 上方往下顯示。

GFNT 點陣字形定位與執行期著色



Figure 27: GFNT 字形定位與點陣圖著色；使用 Cubic-11 及 Unifont 16 的實際「你」字點陣圖